# Software Design Requirements

**for**

# Project CommonGround

**Version 2.X**

**Prepared by Group: WeJustTrynaGraduate (WJTG)**

**<Software Engineering COSC- 4319>**

**<3/27/26>**

# Table of Contents


**1. Designs....4**

**1.1 Macro "Gods-Eye-View".... 4**

1.2 Account Creation....5

Micro Flowchart - Account....5

Jackson - Account.... 6

Object-Oriented Design - Account.... 7

Design Metric Analysis - Account....7

1.3 Listing Creation.... 9

Micro Flowchart - Listing....9

Jackson - Listing.... 10

Object-Oriented Design - Listing....10

Design Metric Analysis - Listing.... 11

1.5 Transaction Creation....13

Micro Flowchart - Transaction....13

Jackson - Transaction.... 14

Object-Oriented Design - Transaction.... 16

Design Metric Analysis - Transaction....16

Black Box Utilization....17

Fan-in.... 18

1.6 Mapping Creation.... 19

Micro Flowchart - Mapping....19

Jackson - Mapping.... 20

Object-Oriented Design - Mapping....21

Design Metric Analysis - Mapping....21

1.7 Feedback Creation.... 24

Micro Flowchart - Feedback....24

Jackson - Feedback.... 25

Object-Oriented Design - Feedback....26

Design Metric Analysis - Feedback....26

1.8 Combined OOD from all designs....28

1.9 Selected design.... 29

**2. Separate data storage....29**

3. N-tier.... 29

4. Race conditions.... 30

5. Deadlocks....31

6. ERD....32

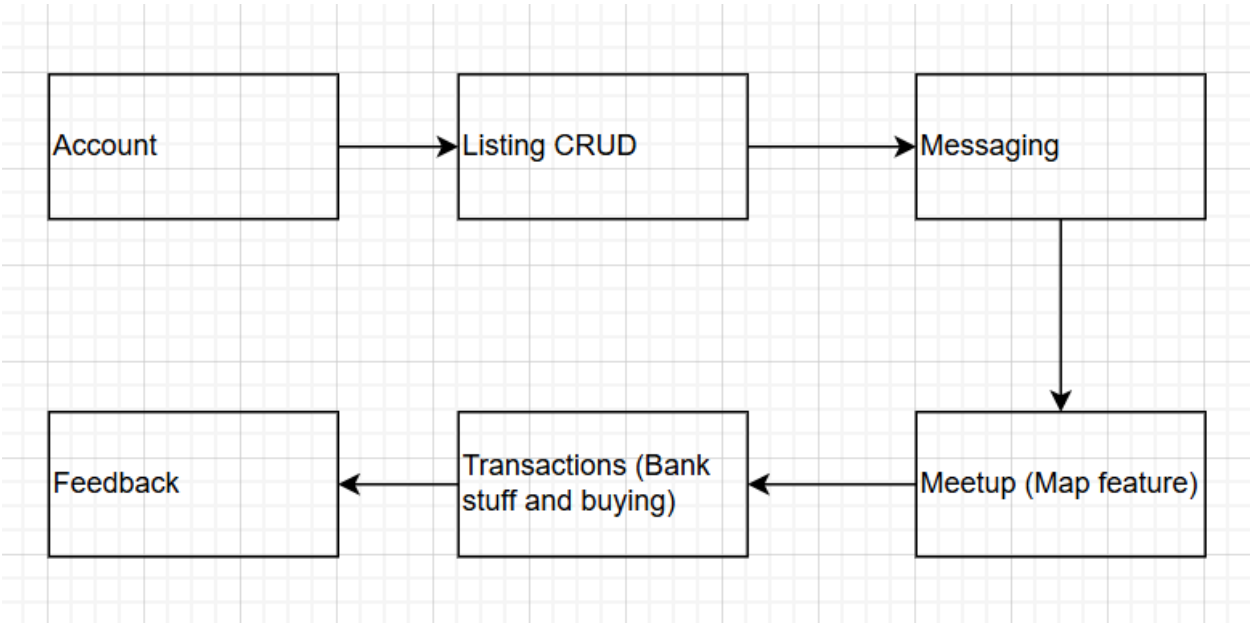
Relational Schema: Project CommonGround ERD Conversion....32

7. Network Security.... 33

8. RPC, Servlets, multithread.....33
9. How the final product will be tested.....34
10. Optimization for space and time.....34
11. Parts in specification that lead to themselves to re-use.....35
12. Implementation plan.....35

# 1. Designs

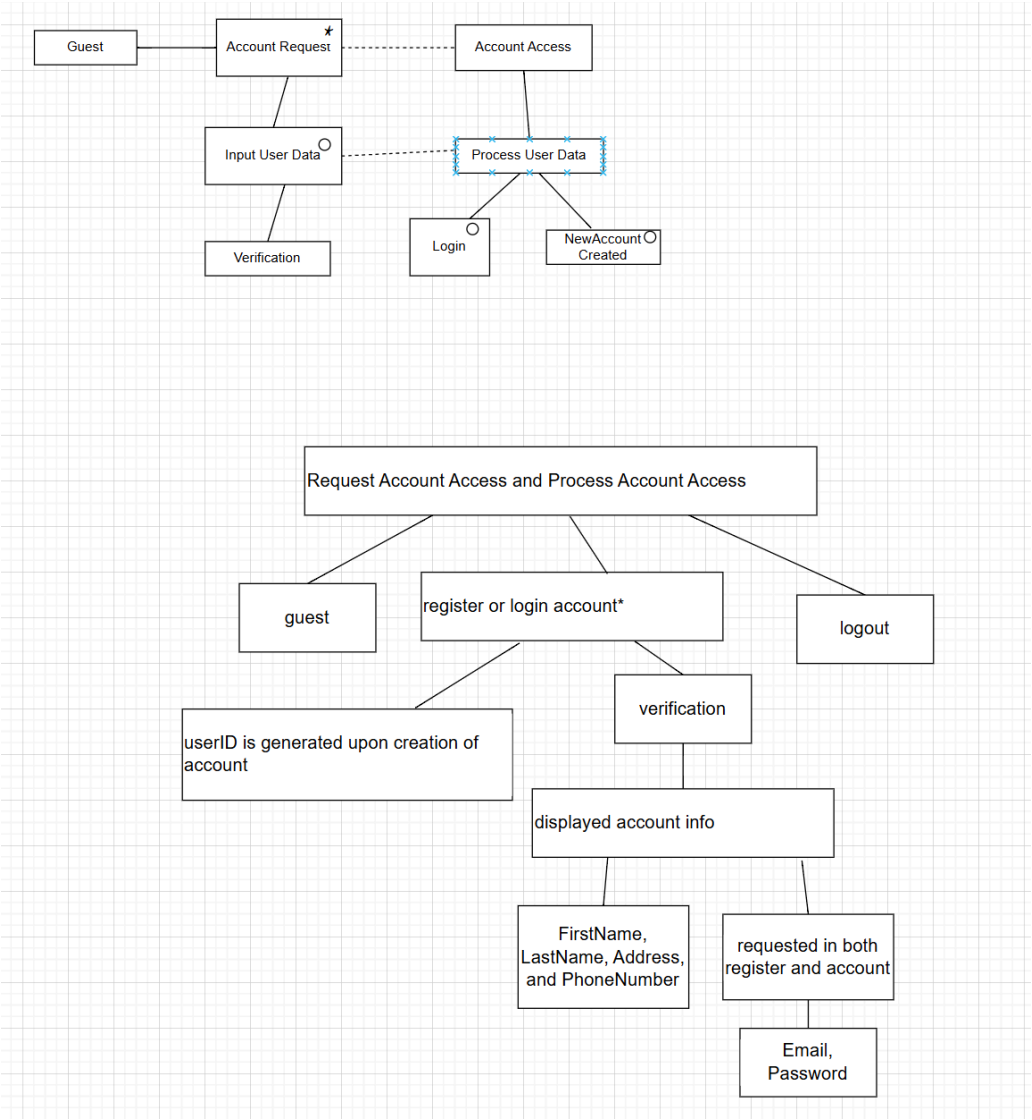
## 1.1 Macro “God’s-Eye-View”



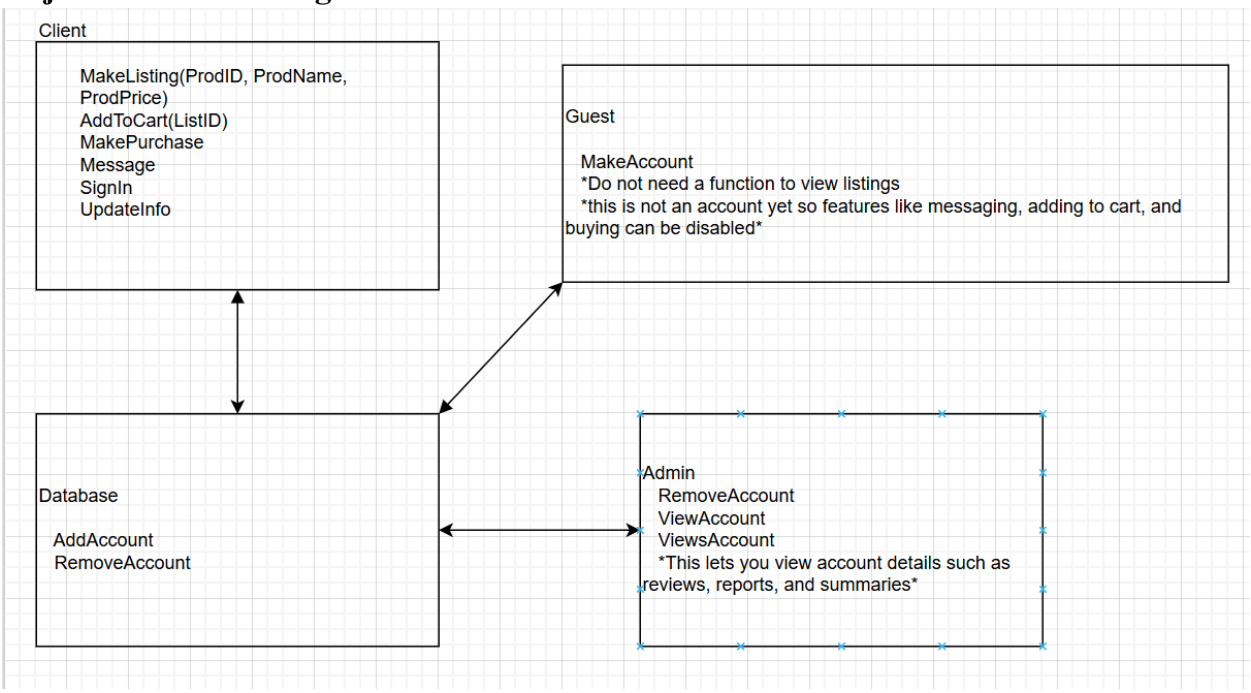
## 1.2 Account Creation

### Micro Flowchart - Account

# Jackson - Account



## Object-Oriented Design - Account



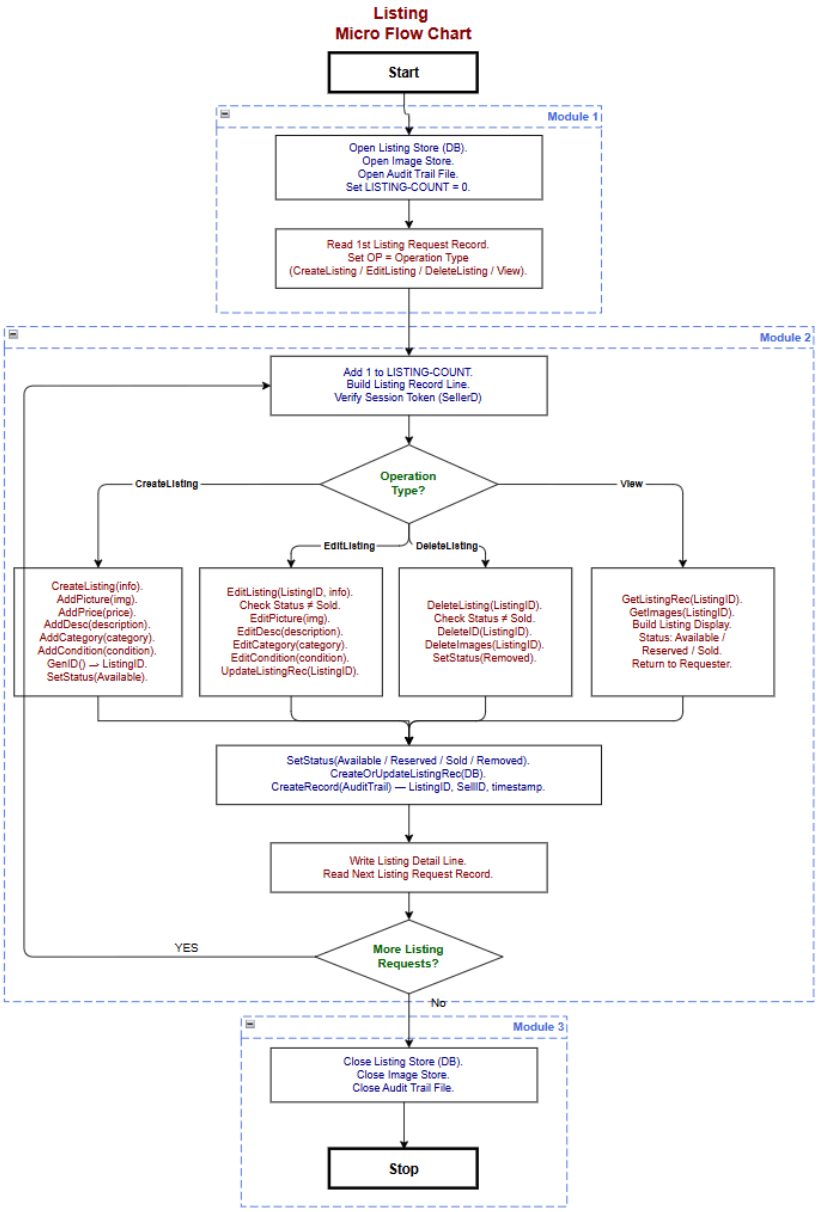

## Design Metric Analysis - Account

| Metric | Analysis |
|---|---|
| Cohesion | All three designs show functional cohesion. The micro flowchart keeps MakeAccount, Sign_In, Update_User_Info, and Remove_Account as self-contained branches with no crossover. Jackson separates the Guest and Login input paths cleanly from the output (NewAccount/Session). OOD achieves the same through the Guest, Client, and Admin class split each class handles only its own responsibility. |
| Coupling (Data) | Micro passes only what each action needs email+password for login, full profile fields for register so data coupling is low. Jackson defines a clear one-way data flow from input fields to output record with no shared state. OOD has the lowest coupling of the three since Database.AddAccount(UserInfo) and RemoveAccount(userID) are the only calls, with no global data shared between classes. |
| Coupling (Control) | All three design techniques produced designs that are low in coupling. Micro routes on ACTION TYPE once, then each branch runs independently |

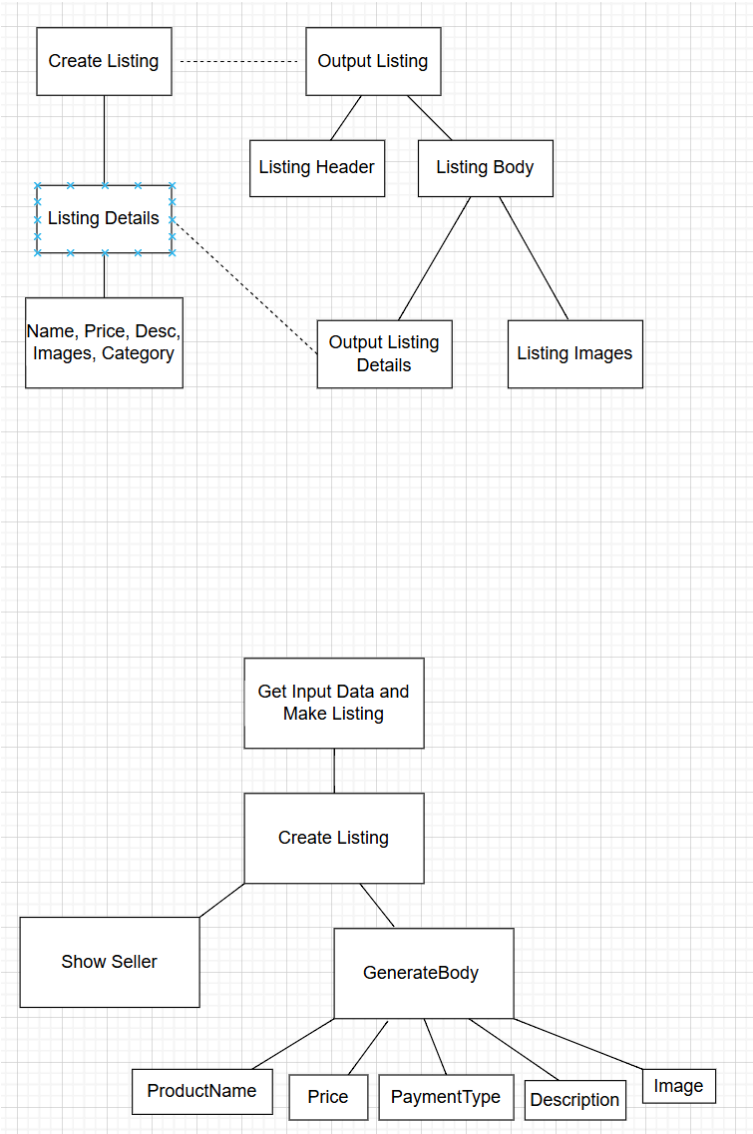
| | |
|---|---|
| | with no flags between them. Jackson's Guest and Login branches are parallel and don't affect each other. OOD methods like Create_Account() and Sign_In() are independent calls no method triggers or controls another. |
| Miller's Law (7±2 chunks) | Micro's MakeAccount has 6 steps (validate email, check duplicate, bcrypt, genID, set role, set status), right within range. Jackson's top level has 2 branches with 3-4 steps each, very clean. OOD has 4 methods in Client's account section and 2 each in Guest and Admin  all well within 7±2. |
| Graicunas' Law (subordinates) | Micro has 4 subordinate action types, each with 4-5 steps, no violations. Jackson has 2 top-level children with manageable sub-steps. OOD has 3 interacting classes (Guest, Client, Admin) plus Database — 4 total relationships, within limits. |
| Factoring | Micro factors well with Module 1/3 handling infrastructure (open/close stores) and Module 2 handling all business logic. Jackson's upper level defines structure (what data), lower level shows processing (how) two clear tiers. OOD factors best: Guest commands, Client commands, Database implements. Upper layers are command and control only. |
| Scope of Effect | Micro's Sign_In only creates a session token; Remove_Account cancels reservations first then deletes  no unintended side effects. Jackson's output is strictly a new account record or a session, nothing else changes. OOD is equally tight; each method only affects Account-related records. All three stay within their intended scope. |
| Scope of Control | All three control the same domain: credential validation, session generation, profile updates, and deletion. Micro enforces this through module boundaries. Jackson enforces it through input/output data structure. OOD enforces it through class encapsulation. |
| Black Box Utilization | Micro treats bcrypt, GenerateSessionToken(), and AuditTrail writes as black boxes. Jackson abstracts verification as a black box step. OOD has the best utilization. The database is fully black-boxed, and password hashing and ID generation are called without exposing internals. |
| Fan-In | Micro and Jackson have moderate fan-in since both produce data that flows into AuditTrail. OOD has the best fan-in  Database.AddAccount is called by both Guest and Client, and the AuditTrail accumulates fan-in from all modules across the system. |

# 1.3 Listing Creation

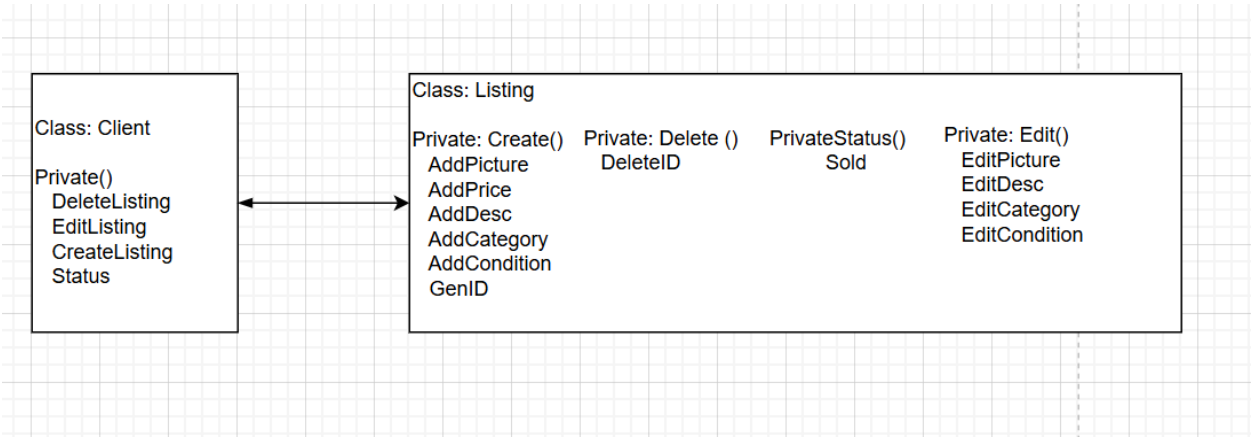
## Micro Flowchart - Listing




## Jackson - Listing




## Object-Oriented Design - Listing



## Design Metric Analysis - Listing

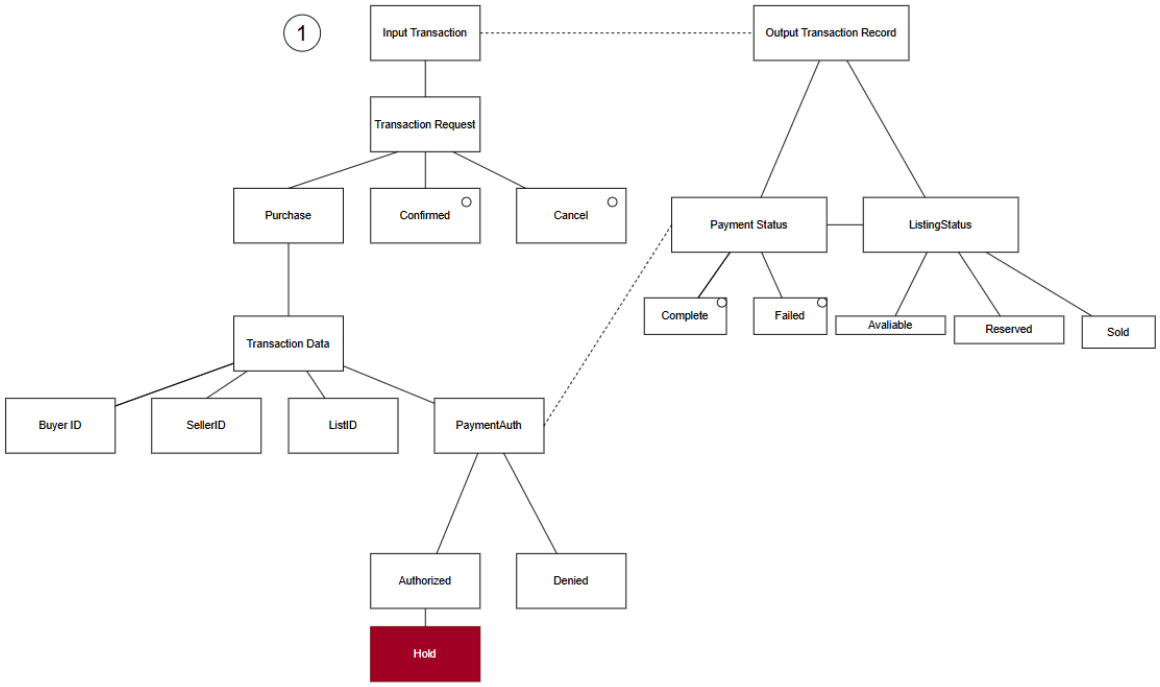
| Metric | Analysis |
|---|---|
| Cohesion | All three are functionally cohesive. Micro keeps Create, Edit, Delete, and View as self-contained branches, each only touching listing data. Jackson's input (Name, Price, Desc, Images, Category) maps directly to output (Listing Body, Header, Images) with no mixing of other concerns. OOD splits responsibility cleanly: Client handles commands, Listing class handles construction, Database handles storage. |
| Coupling (Data) | Micro passes minimal data per operation Create gets all fields, Edit gets ListingID plus changed fields, Delete gets only ListingID. Jackson defines a tight one-way data flow with no shared state between input and output. OOD has the lowest coupling  Client passes ListingID or field data to Listing class, which passes to Database, each hop explicit and minimal. |
| Coupling (Control) | All three are low. Micro's single decision point is OPERATION TYPE; the Edit and Delete branches check Status ≠ Sold as a data guard, not a control flag. Jackson flows straight through with no conditional branching between modules. OOD methods are independent CreateListing() and DeleteListing() don't affect each other |
| Miller's Law (7±2 chunks) | Micro's CreateListing has 7 steps (AddPicture through SetStatus), right at the boundary; Edit has 5, and Delete has 4, both fine. Jackson's input has 5 fields, and the output has 3 branches, very clean. OOD's Client has 4 listing methods and a listing.Create() has 6 subordinate calls, both within range. |
| Graicunas' Law (subordinates) | Micro has 4 operation types at routing, CreateListing approaches the upper boundary at 7 steps but is acceptable. Jackson has 1 input structure and 3 output branches with low interaction count. OOD is a clean 3-level chain (Client → Listing → Database) with 2-4 children per level, no violations. |
| Factoring | Micro factors well with infrastructure in Modules 1/3 and all logic in Module 2. Jackson clearly separates data structure (upper) from processing (lower). OOD factors best  Client is command-only, Listing class handles construction, Database handles persistence. Three distinct responsibilities with no bleed-through. |
| Scope of Effect | Micro's SetStatus(Sold) only updates the listing record; Delete sets status to Removed without touching Transactions. Jackson's output is just a listing record or displayed view, nothing outside the listing store changes. OOD's |

| | |
|---|---|
| | Listing.Status(ListID) only changes that listing no accidental effects on other modules. |
| Scope of Control | All three control listing creation, editing, deletion, and status management only. Micro enforces this through branch isolation. Jackson enforces it through input/output data boundaries. OOD enforces it through class encapsulation, with the race condition check (Status ≠ Sold) scoped inside the Listing class. |
| Black Box Utilization | Micro treats Image Store, AuditTrail.CreateRecord() and GenID() as black boxes. Jackson treats the listing store as a black box destination. OOD has the best utilization: Database, Image Store, GenID, and AuditTrail are all fully black-boxed from the Client's perspective. |
| Fan-In | The level of fan-in for the listing OOD is high when it comes to its features(AddImage(), EditImage()) consistently referring to data for manipulation |

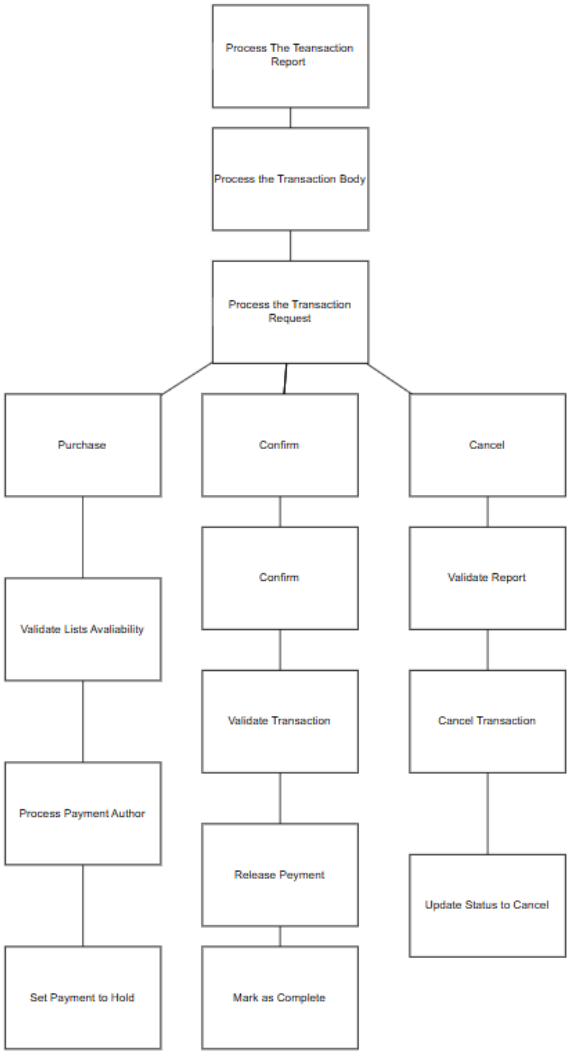
# 1.5 Transaction Creation

## Micro Flowchart - Transaction

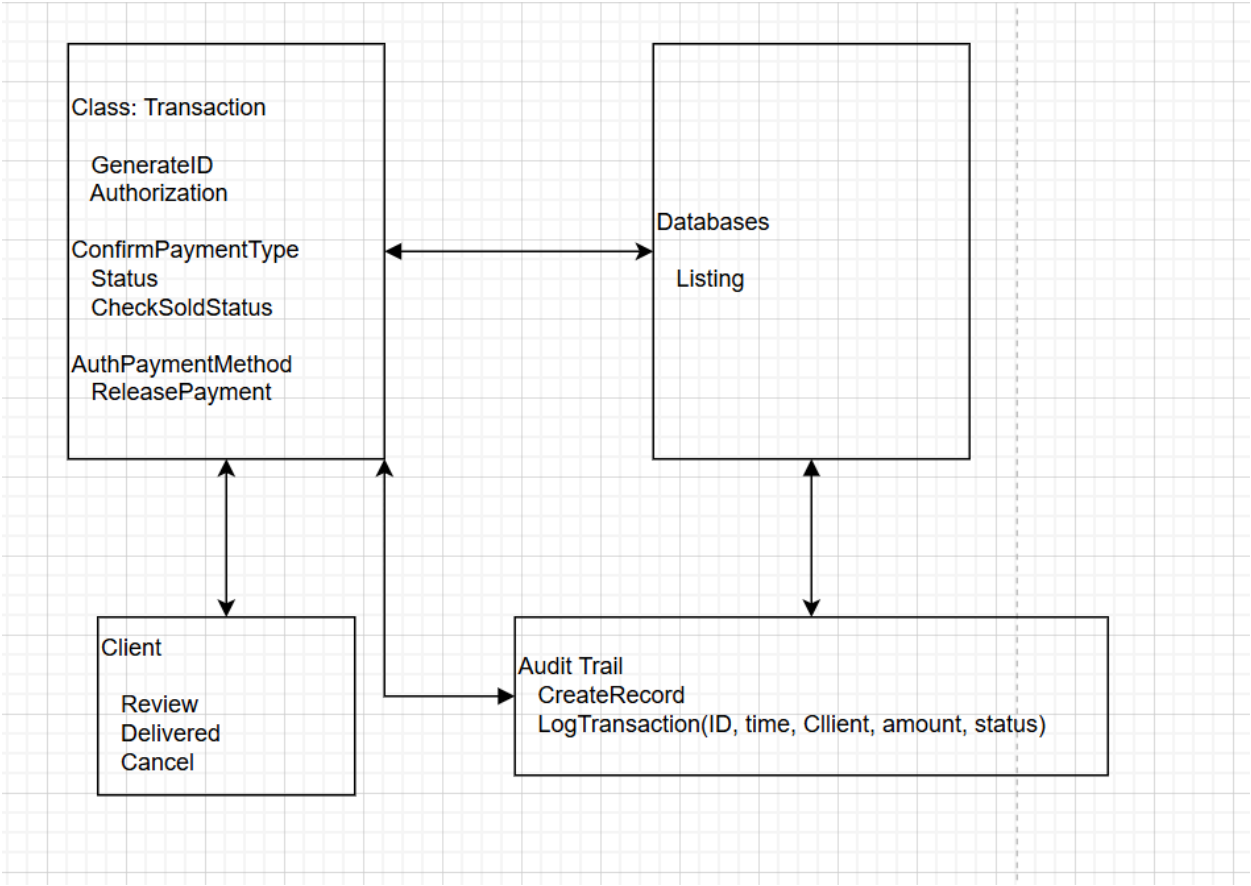
# Jackson - Transaction



2


Process The Teansaction Report
Process the Transaction Body
Process the Transaction Request
Purchase
Confirm
Cancel
Validate Lists Avaliability
Confirm
Validate Report
Validate Transaction
Cancel Transaction
Process Payment Author
Release Peyment
Update Status to Cancel
Set Payment to Hold
Mark as Complete

## Object-Oriented Design - Transaction




## Design Metric Analysis - Transaction

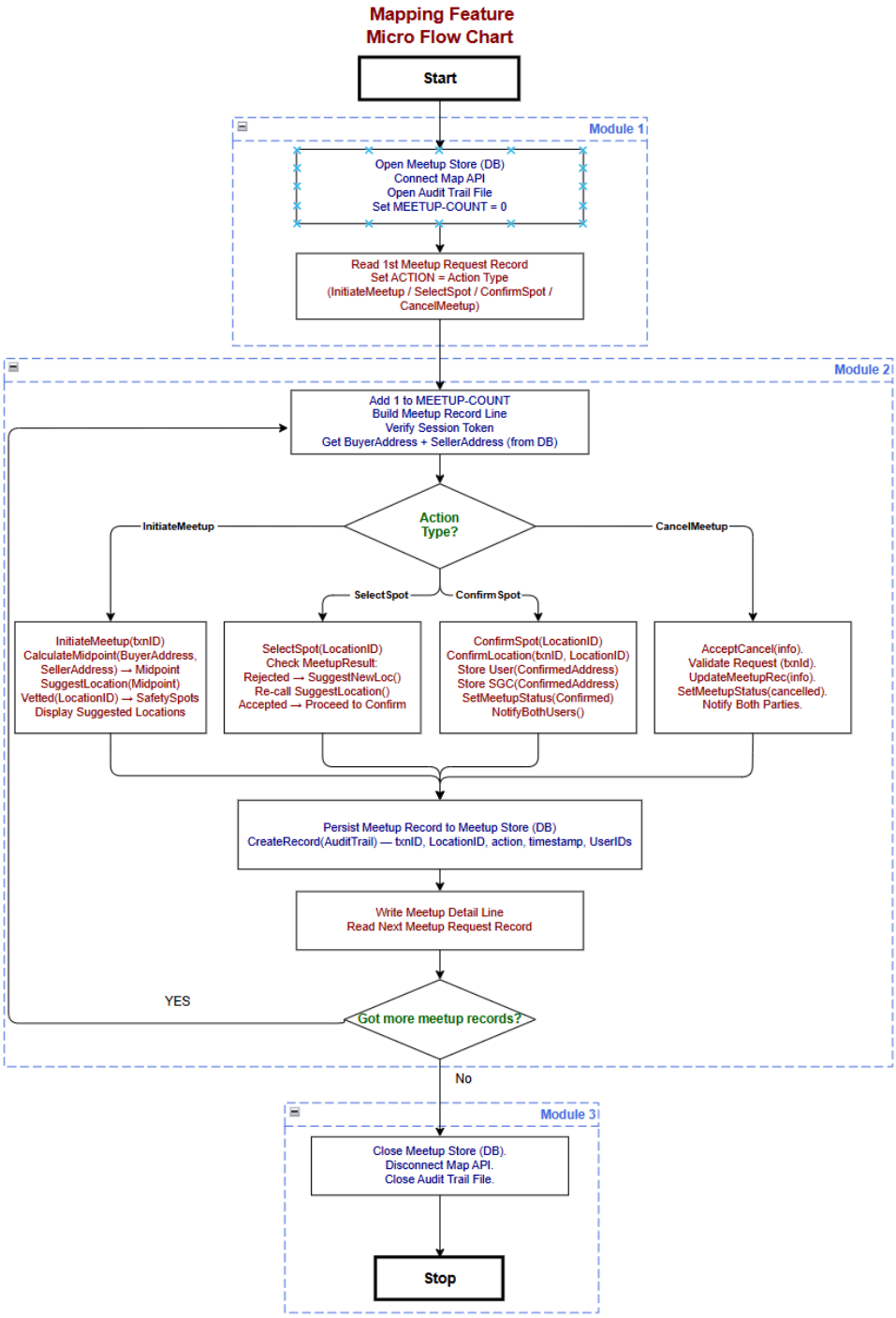
| Metric | Analysis |
|---|---|
| Cohesion | The micro flow chart technique has resulted in easy code readability while also having limited grouping since each function is its own. Jakcsosn's technique resulted in high cohesion as each module had a single, well-defined responsibility. |
| Coupling (Data) | Jackson's technique had resulted in limited data coupling compared to OOD. In the OOD, the data was shared in the transaction object, database object, and audit trail. |
| Coupling (Control) | In micro, each function does not depend on the other since they all provide a different feature. In the OOD control flow was split between the transaction |

| | |
|---|---|
| | object and 2 other objects (Audit Trail and Database). Jackson’s has some slightly high coupling control due to Purchase, Confirmed and Cancel influencing flow control across modules. |
| Miller's Law (7±2 chunks) | Micro follows Miller's law well because it only contains four functions, which is just about the minimum needed. Jackson’s technique is as the number of data structures stays within MIller’s cognitive limit and is easier to maintain and understand. |
| Graicunas' Law (subordinates) | In the OOD technique, it yielded multiple interactions for the Transaction object. Namely, the Client, Audit Trail, and Database have interactions with transactions. This leads to higher system complexity for the OOD design. Jackson’s technique follows Graicuna’s Law by limiting the number of direct relationships each module manages making the system easier to maintain. |
| Factoring | OOD technique resulted in 4 encapsulated objects that represent the entities in the transaction feature. These include the transaction being processed, the database storing the transaction details, and the audit trail that is created from the transaction.<br><br>Micro is split into three distinct modules, which creates coupling.<br><br>Jackson’s technique created 2 data structures that resulted in a single program structure for the transaction feature. |
| Scope of Effect | Micros ReleasePayment() only fires when both confirmation flags are true; cancel resets listing to Available and refunds without touching other modules. Jackson's output is limited to Payment Status or Listing Status changes only. OOD transaction methods only affect Transaction and Listing records, with AuditTrail as a passive write with no unintended side effects across all three designs. |
| Scope of Control | All three control the same domain: listing lock, payment authorization, hold, dual-confirmation, release, and status updates. Micro enforces this through branch isolation and module boundaries. Jackson enforces it through an input/output data structure. OOD enforces it through class encapsulation, with LogTransaction() scoped as the module's own responsibility. |
| **Black Box Utilization** | OOD utilizes a black box to implement the creation of unique transaction IDs. The module has no parameters passed, and no special cases are permitted. |

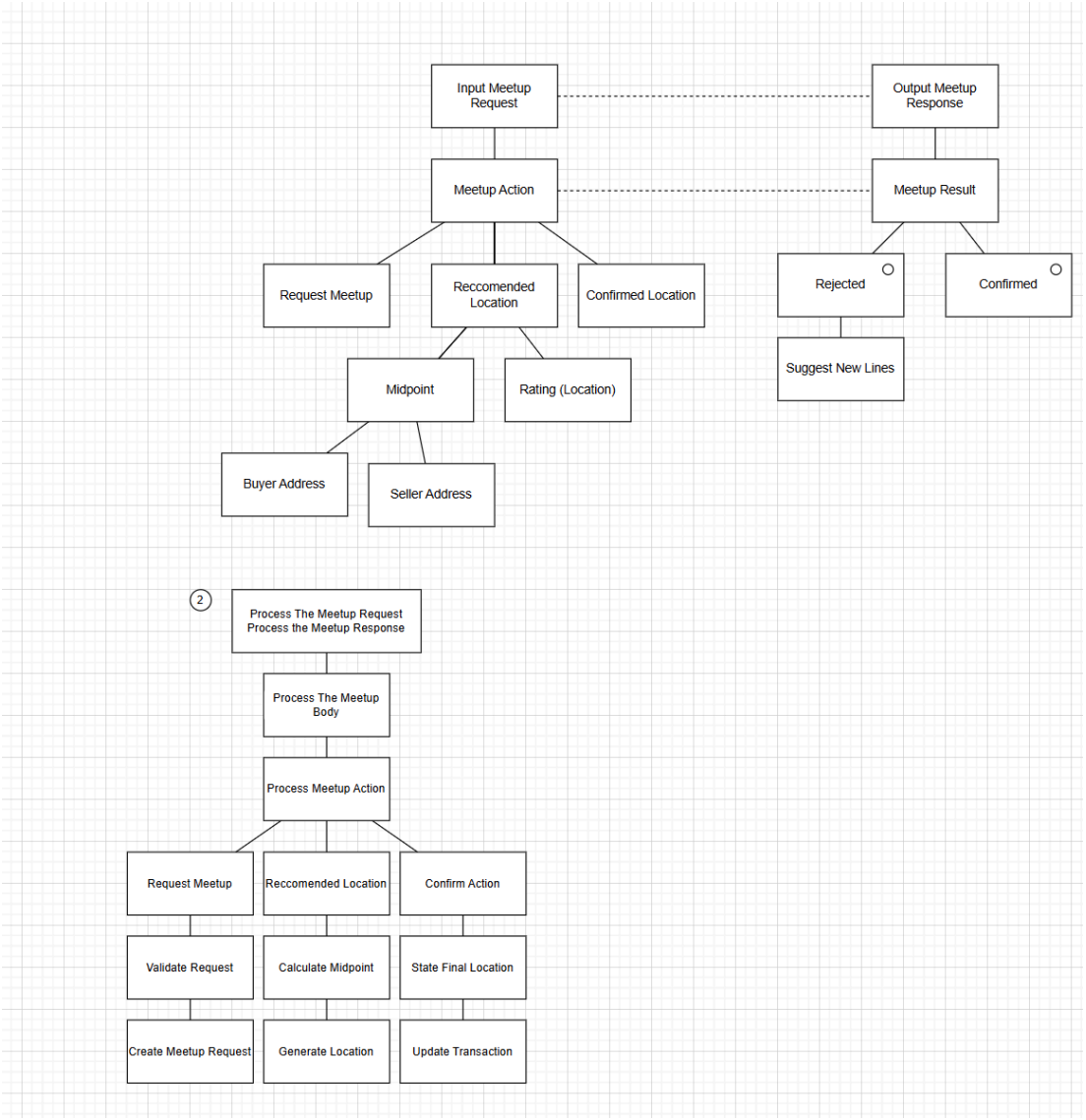
| | |
|---|---|
| **Fan-in** | Transaction OOD creates fan-in whenever there is a need for both users to confirm their sell and then the system confirming that the transaction can go through. Jackson's design technique produced a program structure that had 3 fan-in structures. These were a result of routing the transaction data based on specific user choices. |

# 1.6 Mapping Creation

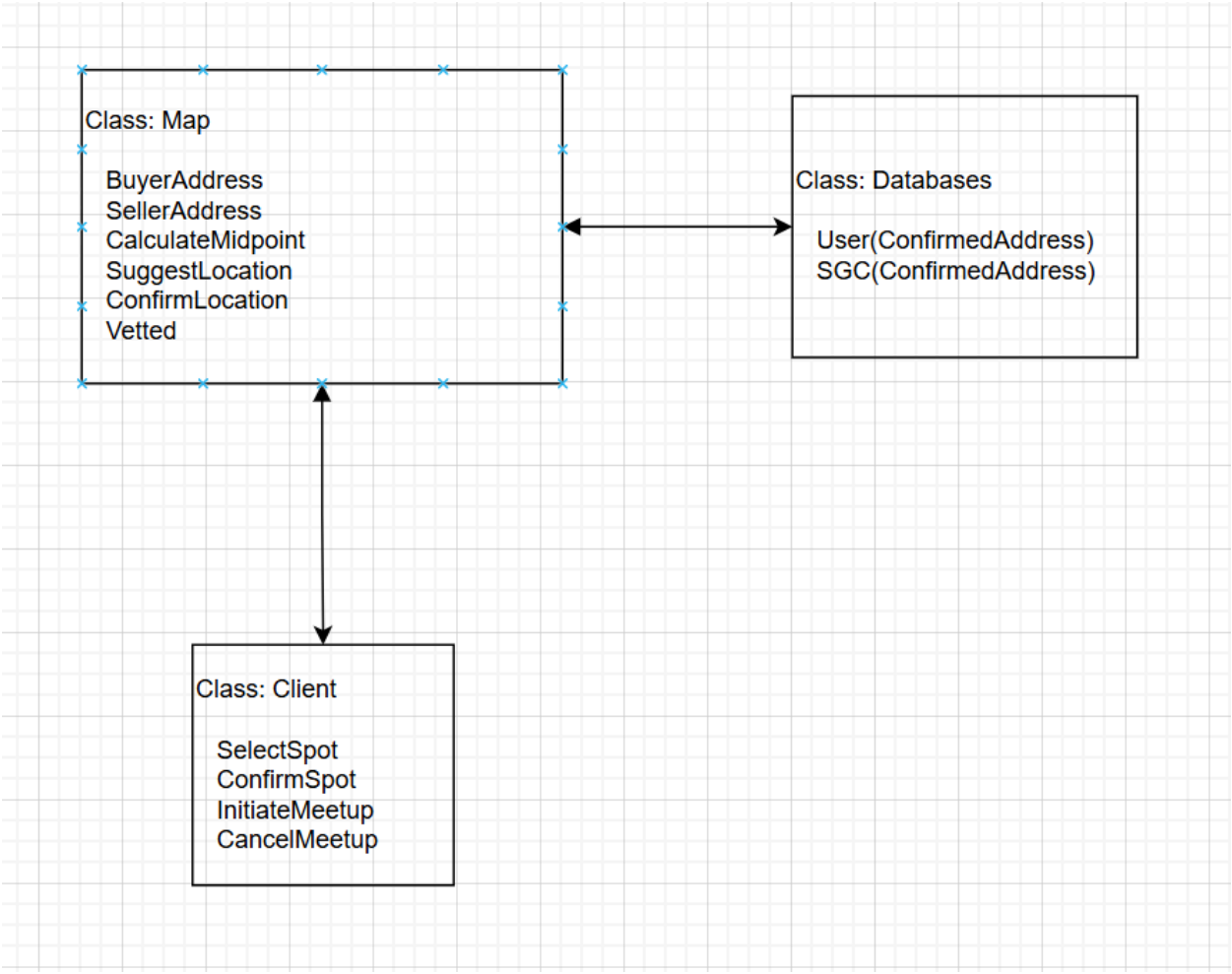
## Micro Flowchart - Mapping



# Jackson - Mapping



## Object-Oriented Design - Mapping




## Design Metric Analysis - Mapping

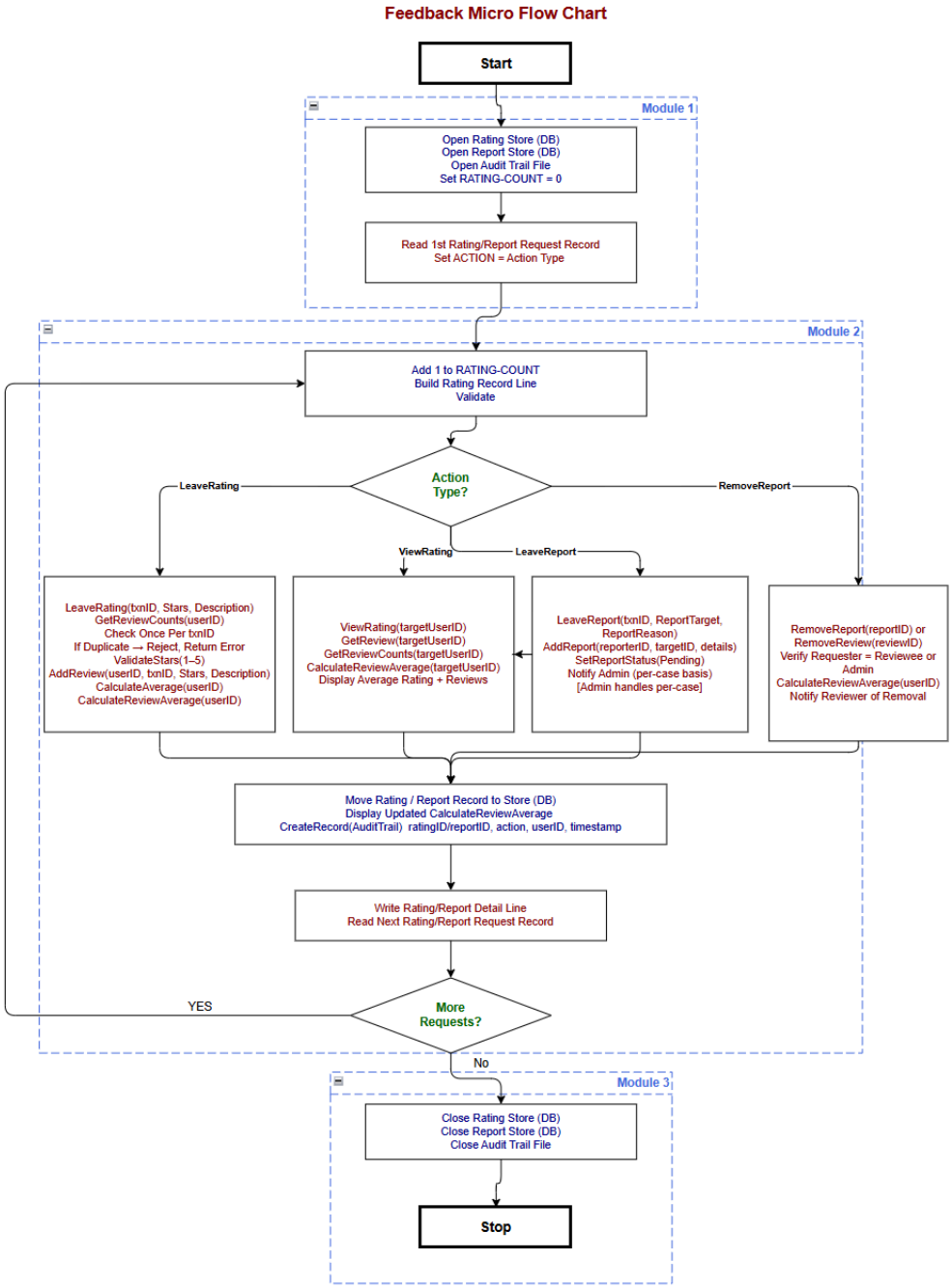
| Metric | Analysis |
|---|---|
| Cohesion | The micro flow chart technique has resulted in easy code readability while also having limited grouping since each function is independent. Jackson's technique resulted in strong functional cohesion as design performs single, well defined tasks such as calculating midpoint, evaluating location, and processing meetup results. |
| Coupling (Data) | The OOD technique resulted in a considerable amount of data coupling as 3 objects utilize the userID and map coordinates. Mirco does not deal with |

| | |
|---|---|
| | data coupling as much since each function has its own features. Jackson's technique resulted in low (good) coupling as modules exchange only necessary information such as buyer and seller addresses to compute midpoint. |
| Coupling (Control) | The OOD design had moderate control coupling as only one object (Client) had a function that would call 2 objects (map/meetup and database) functions. Micro does not deal with control coupling, each function(InitiateMeetup, SelectSpot, ConfirmSpot, Cancel) is independent, and each does its own feature when try to select a meetup spot. Jackson's technique shows low control coupling as each module follows a simple sequence without too many control signals from other modules. |
| Miller's Law (7±2 chunks) | Even though the OOD had moderate control coupling, Miller's law would not be exceeded as each function had only one idea. Although multiple functions could be invoked across multiple objects the idea contained would not vary significantly. Micro does follow Miller's Law by not exceeding the maximum number of ideas. Jackson's technique is as the number of data structures stays within MIller's cognitive limit and is easier to maintain and understand. |
| Graicunas' Law (subordinates) | With each object (3 total) in the OOD having at most 2 interactions with the other modules. Since there are 3 modules there are a total of 6 interactions. Jackson's technique follows Graicuna's Law by limiting the number of direct relationships each module manages making the system easier to maintain. |
| Factoring | Micro factors well its infrastructure in modules ⅓, business logic in module 2, AuditTrail write at the end of module 2 outside the branches. Jackson separates meetup structures(Upper, what data exists) from calculation and storage (lower, how it's structured). |
| Scope of Effect | Micro has small scope of effect since all four functions are all independent and deal with different features(Initiate Meetup, Select Spot, Confirm Spot, Cancel). The OOD had 2 main functions(Calculate_MidPoint(), Suggest_Location()) in the Map object that would be the scope of effect of the Client object function Initiate_Meetup(). Since these functions don't have other functions that call them, the scope of effect remains simple. |
| Scope of Control | Micro has a small scope of control since the four functions deal with a certain function in the meetup process (InitiateMeetup, SelectSpot, ConfirmSpot, Cancel) on getting a meetup spot. Scope of control in the OOD design is localized in the Client object as the function Initiate_Meetup would be the caller of the other 2 objects (Map, Database) and their |

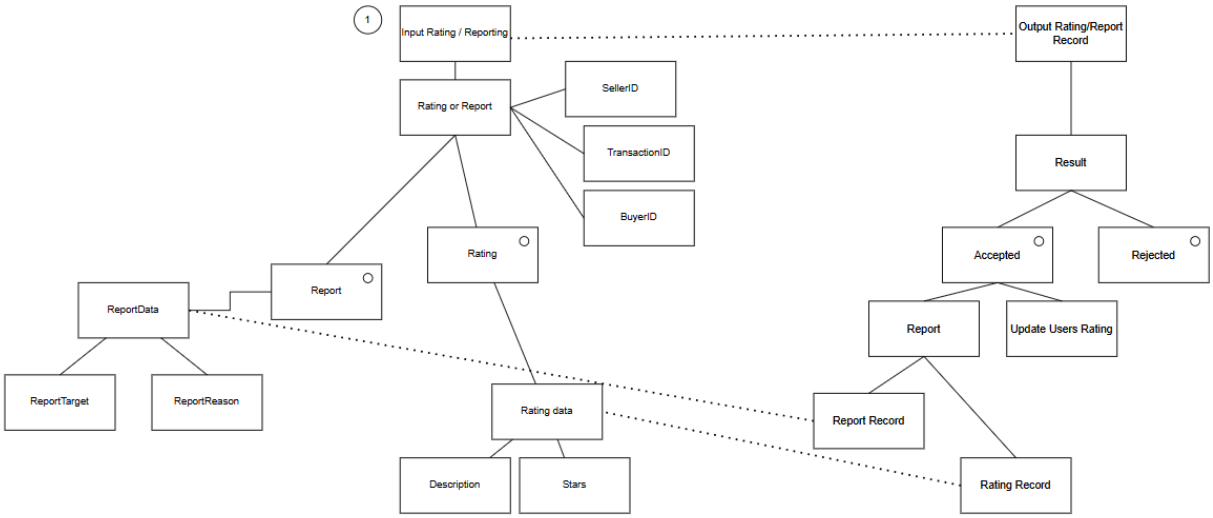
| | |
|---|---|
| | respective functions (Calculate_MidPoint(), Suggest_Location()). |
| Black Box Utilization | Micro treats Map API (SuggestLocation), Vetted() safety check, and NotifyBothUsers() as blackboxes. Jackson gets the location suggestion as a blackbox result but it is less explicit about the external API.OOD has the best utilization, Map API is fully black-boxed through the Map class , Database SGC is a blackbox, the Vetted(LocationID) is a blackbox safety check. |
| Fan-In | OOD forms fan-in in mapping, these appear when calculating midpoint and generating the safe spots for both users to confirm. Jackson's design has no appearances of fan in, but has data structures that consist of location coordinates. These coordinates are then passed through the mid-point calculator module. This is sequentially passed to the user as the middle ground between the two locations. |

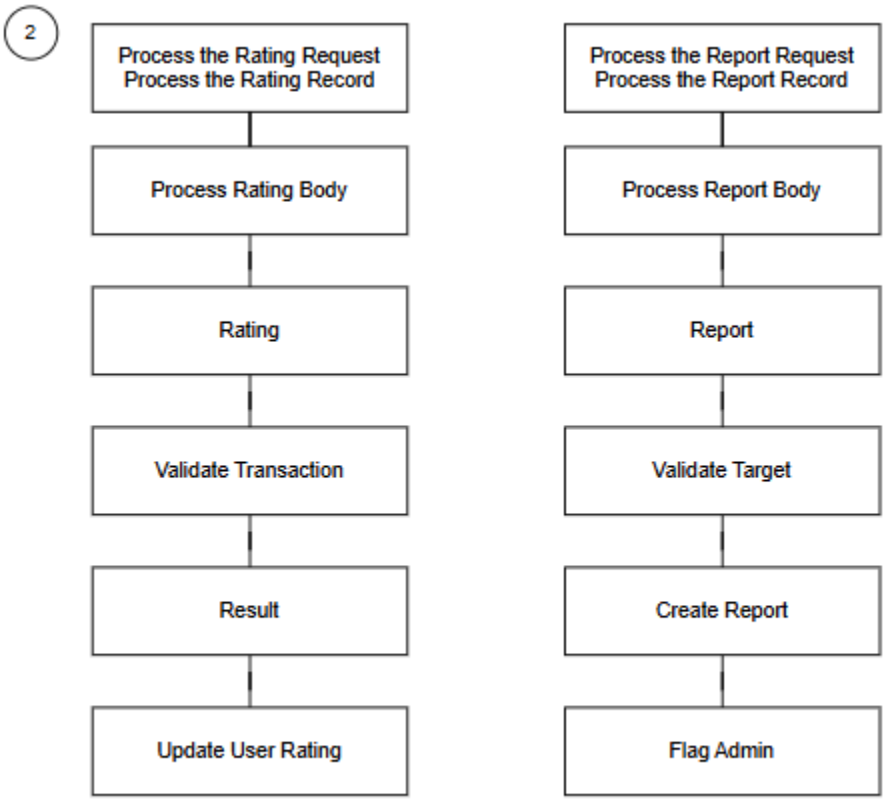
# 1.7 Feedback Creation

## Micro Flowchart - Feedback

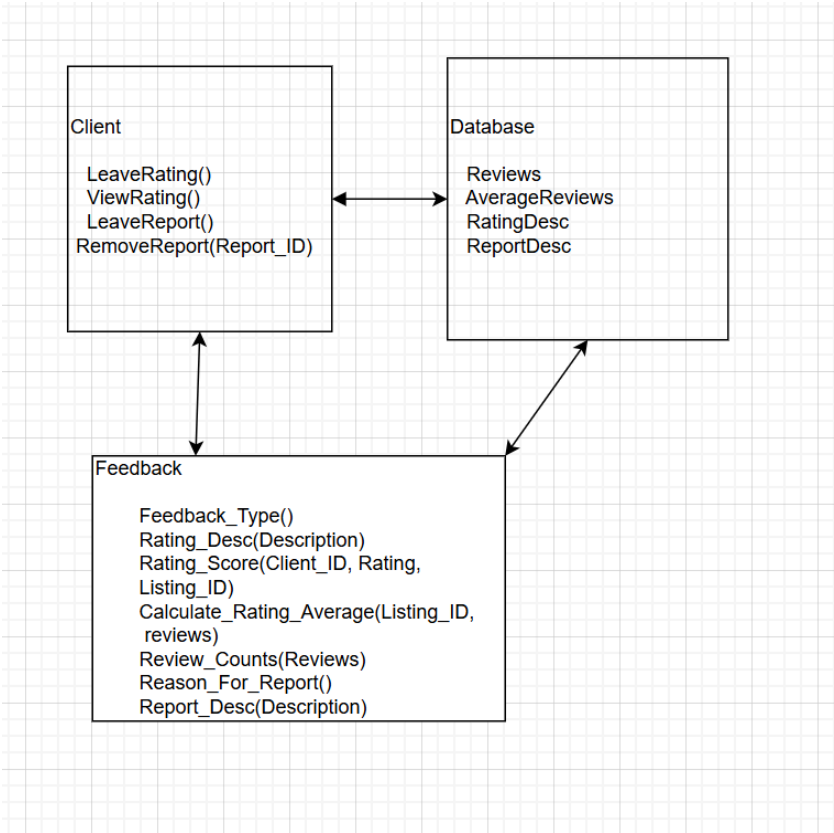


# Jackson - Feedback



## Object-Oriented Design - Feedback

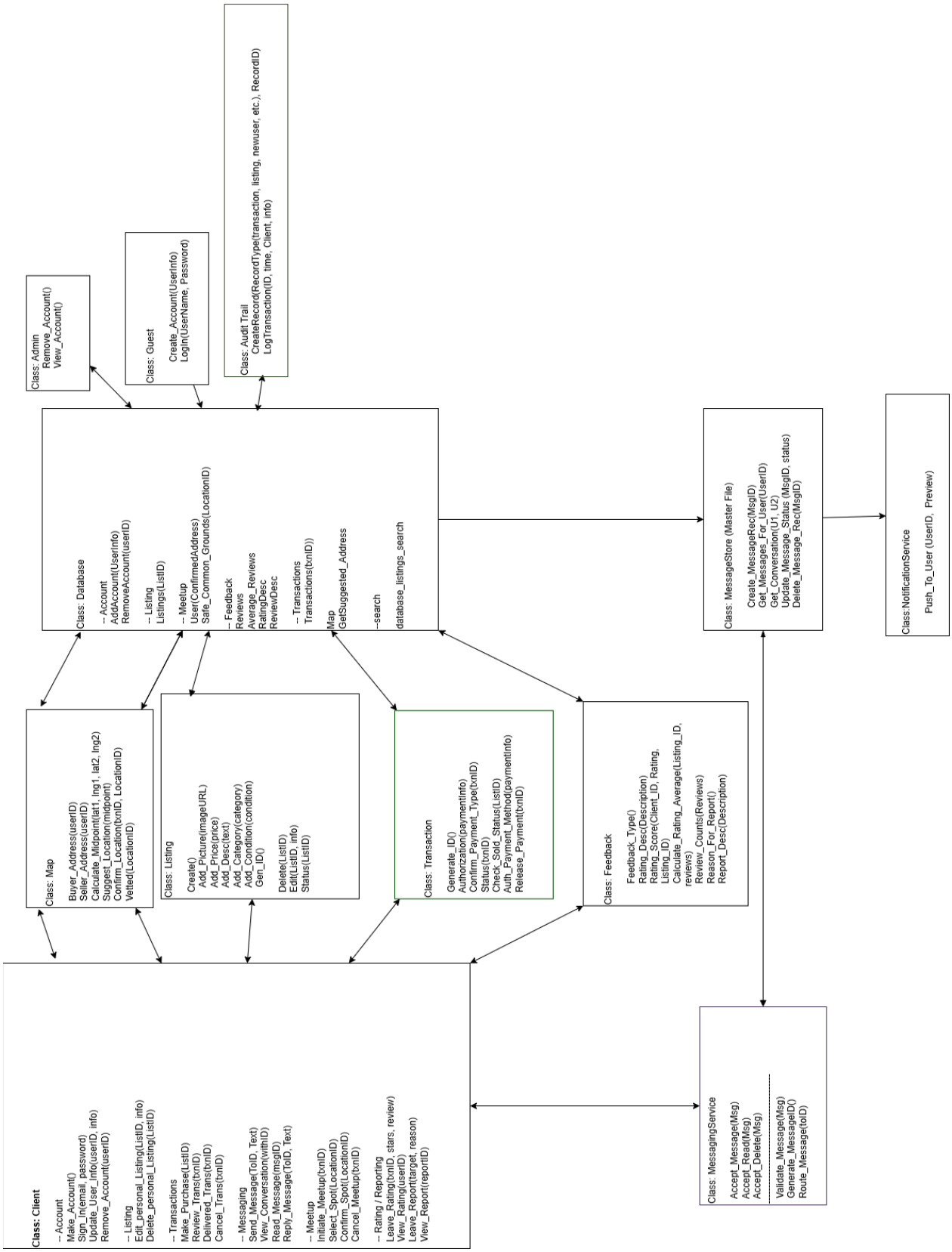



## Design Metric Analysis - Feedback

| Metric | Analysis |
|---|---|
| Cohesion | The cohesion for rating are high since all functions are focused on their own thing. Which ensures each function has a single responsibility. |
| Coupling (Data) | Micro has good data coupling since all the variables in each function are necessary and there are no redundancies. |
| Coupling (Control) | All three eliminate control coupling, Micros one-per-txn rule is enforced by a DB lookup (CheckOncePerTxnID), not a flag. RemoveReport checks permissions through a data query, not a control signal. Jackson methods rating and report branches are fully parallel. OOD client methods are independent calls; Database doesn't pass control signals back to the client. |

| | |
|---|---|
| Miller's Law (7±2 chunks) | The Micro is acceptable under Miller's law as the number of main functions is at most 4. OOD has at most 2 relationships for each object but have back and forth communication with each other. |
| Graicunas' Law (subordinates) | Micro has 4 top-level action types with 4-6 steps each with no violations. Jackson has 2 input and 2 output children with low interaction count. OOD has Client interacting with one object (Database), which has 8 methods but no subordinate classes and a simple two-class structure. |
| Factoring | Micro factors well with infrastructure in Modules 1/3 and AuditTrail write at the end of Module 2 after all branch logic. Jackson's input structures define the data (upper level) and processing steps are lower level  two clear tiers. OOD factors best: Client is purely a command layer, Database handles all computation (GetReview, CalculateAverage, etc.) perfect separation. |
| Scope of Effect | Micro's LeaveRating updates the user's average only; LeaveReport sets status to Pending and notifies Admin, who handles the rest  scope is not overstepped. Jackson's output is limited to a rating record update or a report record only. OOD's AddReview and AddReport only affect their respective tables; CalculateAverage updates the user average; only  all three stay well within scope. |
| Scope of Control | All three control the same domain: rating submission, validation, average recalculation, report creation, and removal. Admin moderation is triggered by this module (Notify Admin) but handles externally correct boundaries in all three. OOD's RemoveReport verifies the requester is the reviewee or Admin before proceeding, enforcing this within the class. |
| Black Box Utilization | Micro treats AuditTrail.CreateRecord(), NotifyAdmin(), and CalculateAverage() as black boxes. Jackson treats rating and report stores as black box destinations but is less explicit about utility calls. OOD has the best utilization CalculateAverage() and CalculateReviewAverage() are black box computations from Client's view, and Database is fully black-boxed. |
| Fan-In | Micro and Jackson have moderate fan-in since Rating and Report stores are written here and AuditTrail gets fan-in from this module. OOD has the best internal fan-in  Database.Get_Review and CalculateAverage are reused within the module for both LeaveRating and RemoveReport, reducing redundancy. |

# 1.8 Combined OOD from all designs



## 1.9 Selected design

Object Oriented Design is the best option since it has lower coupling compared to micro flow chart and MA Jackson. It also reduced system interaction complexity (Graciunas' Law) by encapsulating related functions into objects that conveyed the interactions between the related functions. While macro-micro typically has reduced fan-out leading to a more sequential process, Object Oriented Design makes up for it on average by performing best on implementing black boxes which will help with developing segments in the future. The design translation to implementation is also simpler to do as each functionality can simply be made into its own file with its corresponding classes and functions. While micro/macro flow charts aid in creating a step-by-step process on each function's job it doesn't accurately show the relationships between them in the full system design.

# 2. Separate data storage

We are using relational databases as our storage element. All persistent data including user accounts, listing, transactions, meetup records, messages, and Feedback are stowed in separate normalized tables. Every table is in third normal form meaning each table as a primary key, all non-key attributes depend only on the primary key, and there are no transitive dependencies. For example the listing table stores a Seller ID as a foreign key back to the User table rather than repeating the seller name or contact info inside every listing record. This prevents update anomalies so if a seller changes their info it only has to be updated in one place. The Rating table uses a composite key of Transaction ID and Reviewer ID to enforce the one-rating-per-transaction rule directly at the data level. The database is accessed exclusively through the Database class in our OOD design so every other module treats storage as a black box and never queries it directly.

# 3. N-tier

CommonGround will use a closed N-Tier architecture style. It will consist of three tiers total. A transaction will begin at the top tier and will be the Presentation tier. This consists of the user interface, which will be a website hosted through an Apache server. This allows the clients to browse the listings, initiate transactions, and message other clients to coordinate said transactions. For example, when a client wants to initiate a transaction, this information will be sent to the middle tier. The presentation tier calls directly to the middle tier and has no direct access to the database.

The middle tier is where all of the data gets processed and is the core part of CommonGrounds functionality. The "Brain" of the program exists here. Java will be used in this tier, matching the object oriented nature of the project's chosen design technique. Any client inputs through the

presentation tier relies on the business logic held here. After the user initiates a transaction, this layer will be responsible for things like checking for race conditions and calling to the database to locate a Common Ground for the user. This tier calls directly to the bottom tier as data is processed and modified.

The bottom tier is the data tier. It responds directly to the middle tier. This includes the database and where management of the database is performed. Retrieval of data, its storage, and backups are also held in this tier. Any changes that are made through the logic of the middle tier will reflect here. Once the data is updated, it will send it back up through the middle tier and to the Presentation tier so the user can also see the reflected changes. The database will be held using MySQL.

# 4. Race conditions

The main race condition risk in CommonGround is two buyers trying to purchase the same listing at the same time. Both could check the status, see "Available," and both attempt to reserve it simultaneously  leaving the listing in an inconsistent state or double-booked.

To prevent this, when a buyer initiates a purchase the system performs an atomic check-and-reserve using a database-level row lock (SELECT ... FOR UPDATE). This means only one thread can read and update that listing record at a time. If Buyer A acquires the lock first and sets the status to "Reserved," Buyer B's request comes back with a status of "Not Available" and the transaction is aborted before payment is ever authorized. The listing lock is held only for the duration of the status check and update, then released immediately this is AcquireListingLock(ListID) / ReleaseListingLock(ListID) shown explicitly in our micro flowchart.

A second race condition exists when a seller tries to edit or delete a listing at the same time a buyer is purchasing it. The system handles this the same way Edit and Delete both check Status ≠ Sold/Reserved as an atomic read before proceeding. If a purchase is in progress and the listing is locked, the edit or delete request is rejected with an error.

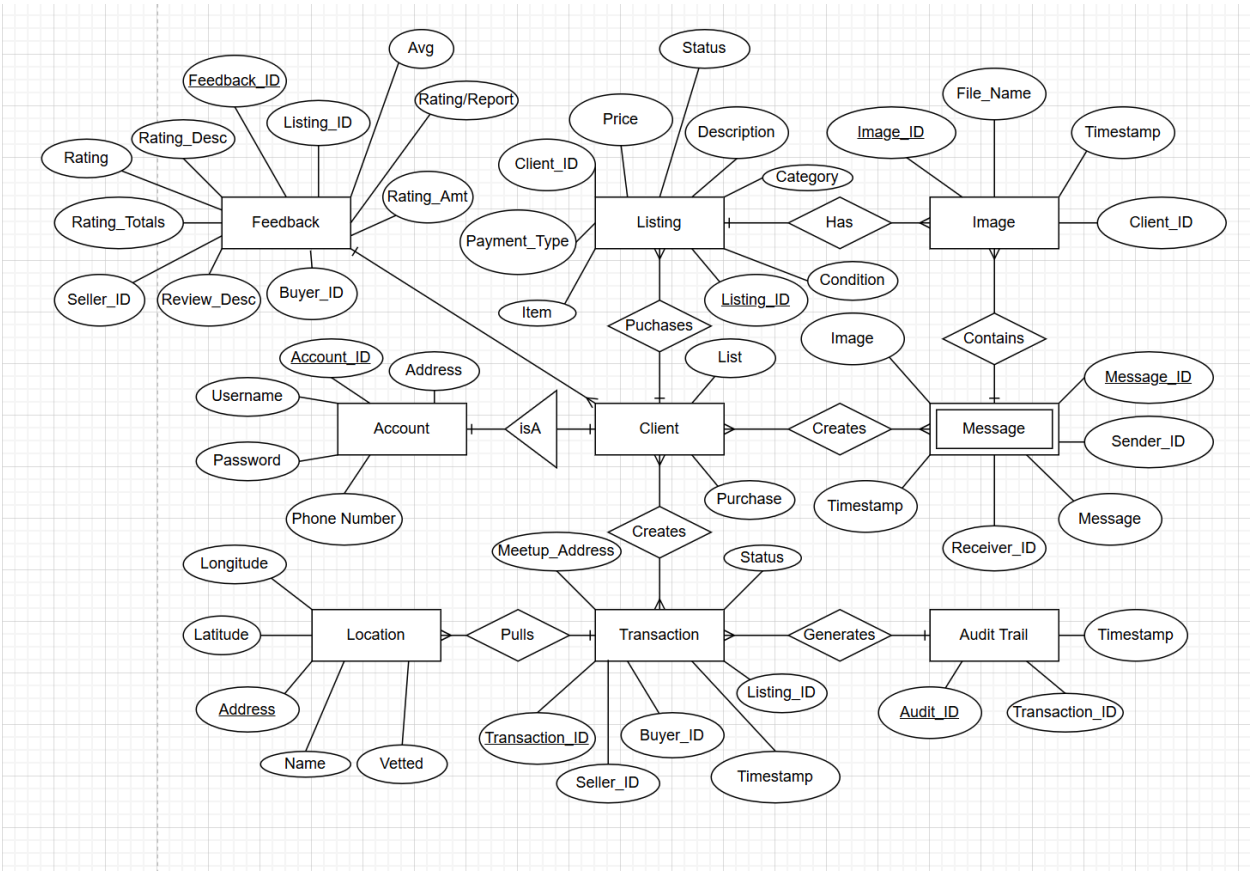
A third case is duplicate purchase requests from a buyer clicking the purchase button multiple times. Our specification addresses this the system checks for an existing active transaction on that listing before creating a new one, preventing duplicate payment authorizations.

# 5. Deadlocks

A deadlock requires at least two related records that separate processes compete over. In CommonGround the classic scenario is a buyer initiating a purchase and a seller cancelling or editing their listing at the same time. Process 1 holds a lock on the Listing record and is waiting to write to the Transaction record. Process 2 holds a lock on the Transaction record and is waiting to update the Listing record to reset its status. Both are asleep waiting on each other and neither can proceed, which is the circular wait condition. We avoid this three ways. At the database level, MySQL detects deadlocks automatically by monitoring lock wait graphs. When it finds a circular wait it kills the younger transaction, rolls it back, and returns an error to the application which then retries.

This is essentially the WAIT-DIE method where older transactions get priority. At the programming level, we always acquire locks in the same fixed order across all operations, Listing first then Transaction. Linear ordering like this prevents circular wait from ever forming in the first place since no process will ever hold a Transaction and wait on Listing if everyone always requests Listing first. For scheduling, any operation that calls an external service like payment authorization runs with a timeout. If the thread does not commit within the time limit it rolls back automatically and releases its locks, which prevents the hold-and-wait condition from sitting open indefinitely while waiting on a third party response.

# 6. ERD




## Relational Schema: Project CommonGround ERD Conversion

Account (int: Account_ID, string: Username, string: Password, int: Phone Number, string: Address)

Client (boolean: List, boolean: Purchase, int: Account_ID) #Account_ID references (Account_ID)

Transaction (int: Transaction ID, int: Buyer_ID, int: Seller_ID, int: Listing_ID, int: Timestamp) # Buyer_ID FK references Client(Account_ID) Seller_ID FK references Client(Account_ID)

Location (string: Address, int: Latitude, int: Longitude, string: Name, boolean: Vetted)

Audit Trail (int: Audit_ID, int: Transaction_ID, int: Timestamp) #Transaction_ID FK references Transaction(Transaction_ID)

Message (int: Message_ID, int: Sender_ID, int: Receiver_ID, string: Message, int: Timestamp, jpeg: Image) #Sender_ID FK references Client(Account_ID) Receiver_ID FK references Client(Account_ID)

Image (int: Image_ID, int: Client_ID, string: File Name, int: Timestamp) #Client_ID FK references Client(Account_ID)

Listing (int: Listing_ID, int: Client_ID, string: Payment_Type, int: Price, string: Description, string: Category, string: Item, int: Status, int: Condition) #Client_ID FK references Client(Account_ID)

Feedback (int: Feedback_ID, int: Buyer_ID, int: Seller_ID, int: Listing_ID, int: Rating_Totals, int: Rating_Amt, float: avg Rating_Totals/Rating_Amt, int: Rating, boolean: Rating/Report, string: Rating_Desc, string: Review_Desc) # Buyer_ID FK references Client(Account_ID) Seller_ID FK references Client(Account_ID) Listing_ID FK references Listing(Listing_ID)

# 7. Network Security

For network security in the project the system will use both TLS and public/private key encryption. The server will generate a private key and a corresponding public key using an outside tool (possibly OpenSSL). The public key will be shared with client applications, while the private key will stay on the server. Clients will use the public key when encrypting sensitive transaction data before sending it. Communication will also occur over HTTPS to ensure encryption during transmission. Passwords are also hashed using bcrypt with a unique salt per user.

# 8. RPC, Servlets, multithread

RPC is a way for a client to call a function on a remote server as if it were local. It works well for tightly coupled systems but requires more infrastructure setup like stubs and skeletons, which adds complexity we don't need. Multithreaded sockets give you raw control over the communication protocol and scale well using the pattern of spawning a new server thread per client connection, but that means we'd have to manage threads manually which is a lot of extra work on top of everything else. Servlets are the best fit for CommonGround. They handle HTTP natively, integrate cleanly with HTTPS and TLS, support session management out of the box, and the servlet container manages a thread pool for us so we get multithreading without having to implement it ourselves. For the hold-payment feature specifically, we'll spawn a background thread from the servlet that monitors the transaction state and releases payment once both buyer

and seller confirm this follows the producer-consumer pattern. We're building a web application so Servlets are the natural choice.

# 9. How the final product will be tested

The final product will be tested by creating two accounts, creating a listing with one of them, and making a purchase with the other. Since both accounts will be new, they will each need to go through the Account Creating Process. The buyer will need to input their payment details and address and the lister will need to input their deposit details and address before a transaction can be completed. For testing purchases, this demonstration will only show payment through the app. After the transaction is complete, the buyer will have a chance to review the seller. Finally, the buyer will delete their account. This will successfully demonstrate the primary goals of our software.

Measurements for Success

- Be recommended at least one CommonGroundSpot for a given transaction.
- Demonstrate what will happen if a charge is declined.
- Perform a successful transaction with payment through the app, including demoing the 'hold payment' feature.
- Seller rating populates their account after the transaction.
- When a user requests account deletion, they receive an email and the visual feedback shows.

# 10. Optimization for space and time

Space and time optimization are generally mutually exclusive goals so we prioritize based on where each matters most for our system. For space, the biggest concern is listing images. We are not storing images as binary blobs in the database because that would bloat it fast and kill query performance. Instead images get stored in a file system or object storage and only the file path is saved in the Listing record. Listings that are sold or removed get soft deleted and moved to an archive table so the active listings table stays small and queries stay fast.

Our OOD design supports this well since the Listing class and Database class are decoupled, meaning we can swap in an external image store without touching the rest of the system. High cohesion and low coupling is what makes packaging decisions like this possible after the design is done. For time, the most performance sensitive operations are listing search and purchase initiation. We will put indexes on the Listing table for status and category since almost every query filters on one or both of those. The midpoint calculation for meetup is done in memory in the logic tier so there is no extra database call for that. The Map API call is our slowest external

dependency so it runs asynchronously and does not block the UI while locations load. AuditTrail writes are also async and happen after a transaction completes rather than before so logging does not add to the user facing response time.

# 11. Parts in specification that lead to themselves to re-use

Audit trail can be reused in different settings for other applications. Programs like Finance & Banking for recording every transaction (creation, approval, modification) for accountability. For IT Security/Intrusion Detection when Logging user login/logout times and network connection attempts to identify unauthorized access. The messaging system can be repurposed for Customer Support in Businesses that use instant messaging (e.g., WhatsApp Business, Messenger) to provide real-time service and resolve issues. Also can be used in Internal Communication programs that teams use platforms like Slack or Microsoft Teams for instant communication, file sharing, and project updates. Finally the Account creation can be reused in any different program or application that requires having a specific different account per user like most social media platforms.

# 12. Implementation plan

**March 25–27 (right now)** — Design due March 27th before 8am
**March 27–30** — Set up the project environment
**March 30 – April 6** — The three high-priority features from the spec (Account, Listing CRUD, Transactions). Transactions is the biggest chunk; it has the race condition lock, dual confirmation logic, and idempotency key  so it gets more days.
**April 7–14** — Meetup/Map, Messaging, and Feedback run in parallel since they're independent modules. These are the secondary priority features but still extremely necessary for the project.
**April 10–16** — Security hardening and concurrency testing overlap with feature work. The race condition and deadlock tests need to run before acceptance. Security is needed but not the end of the world if it doesn’t work before the final day.
**April 15–19** — Left over testing and polishing fixing any errors.
**April 19–20** — Due date crunching.